CYC475 - Topics in Cyber Security (Web Technologies and Programming)

WEEK1: CATEGORIES OF WEB APPLICATIONS, WEB ARCHITECTURES

NAJLA RAZA – LECTURER

COMSATS UNIVERSITY, ISLAMABAD CAMPUS

Lecture Contents

- Categories of Web Applications
- Web Architectures

Categories of Web Applications

- Static Web Applications
- Dynamic Web Applications
- Single Page Applications
- Multi-page Applications

Static Web Applications

- Pre-rendered HTML pages
- Delivered to the user's browser as is.
- These applications do not require server-side processing for each request.
- Examples: Portfolio websites, company information sites, documentation, disaster report.

Dynamic Web Applications

- Generate content dynamically based on user interaction or other factors.
- These applications rely on server-side processing to render HTML.
- Examples: E-commerce sites, social networks, content management systems (CMS).

Single Page Applications

- Load a single HTML page and dynamically update content as the user interacts with the application.
- Dynamic framework that covers the interaction with the user by the updated data within the existing page without loading totally new pages from the server.
- Examples: Gmail, Google Maps.

Multi-page Applications

- Load multiple web pages.
- Each user interaction or page request results in a new HTML page being fetched and displayed.
- Examples: Traditional e-commerce sites.

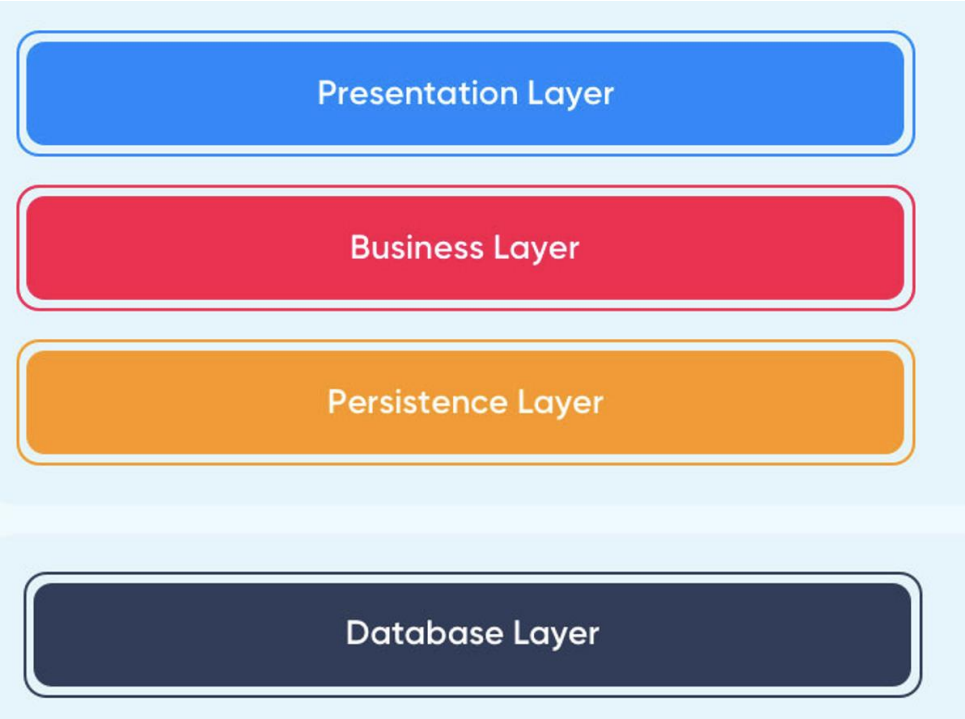
Web Architectures

- Layered Architecture
- One-tiered architecture
- Two-tier architecture
- Three-tiered architecture
- N-Tiered architecture
- Monolithic architecture
- Microservices architecture

Layered Architecture

- A traditional approach where the application is divided into layers, each responsible for a specific aspect of the application. Common layers include:
- **Presentation Layer**: Handles the user interface and user experience (UI/UX).
- **Business Logic Layer**: Implements the core functionality and rules of the application.
- **Data Access Layer**: Manages data storage, retrieval, and manipulation.
- **Advantages**: Modularity, separation of concerns, easier maintenance.
- **Use Cases**: Standard business applications, where clear separation of logic, data, and presentation is required.

Layered Architecture



One-tiered architecture

- The entire application, including the user interface, business logic, and data access code, is bundled into a single software package.
- Advantages: Simple deployment, easy to develop for small applications.
- Disadvantages: Hard to scale, difficult to maintain as the application grows.
- Use Cases: Small, simple applications or prototypes.

Two-tiered architecture

- Splits the application into two layers:
  - Client Tier: Runs on the user's machine and interacts with the server.
  - Server Tier: Handles data processing, business logic, and database operations.
- Advantages: Reduces server load, easier to manage compared to one-tier.
- Disadvantages: Limited scalability, complex client-side code.
- Use Cases: Desktop applications with a client-server model, small to medium-sized web applications.

Three-tiered architectures

- Adds an additional layer to the two-tier model, resulting in three layers:
  - Presentation Tier: The front-end, usually running in a web browser.
  - Application Tier (Logic Tier): Manages the business logic and processes.
  - Data Tier: Manages database interactions.
- Advantages: Better separation of concerns, scalability, and maintainability.
- Disadvantages: More complex to develop and manage.
- Use Cases: Enterprise-level applications, where scalability and separation of layers are crucial.

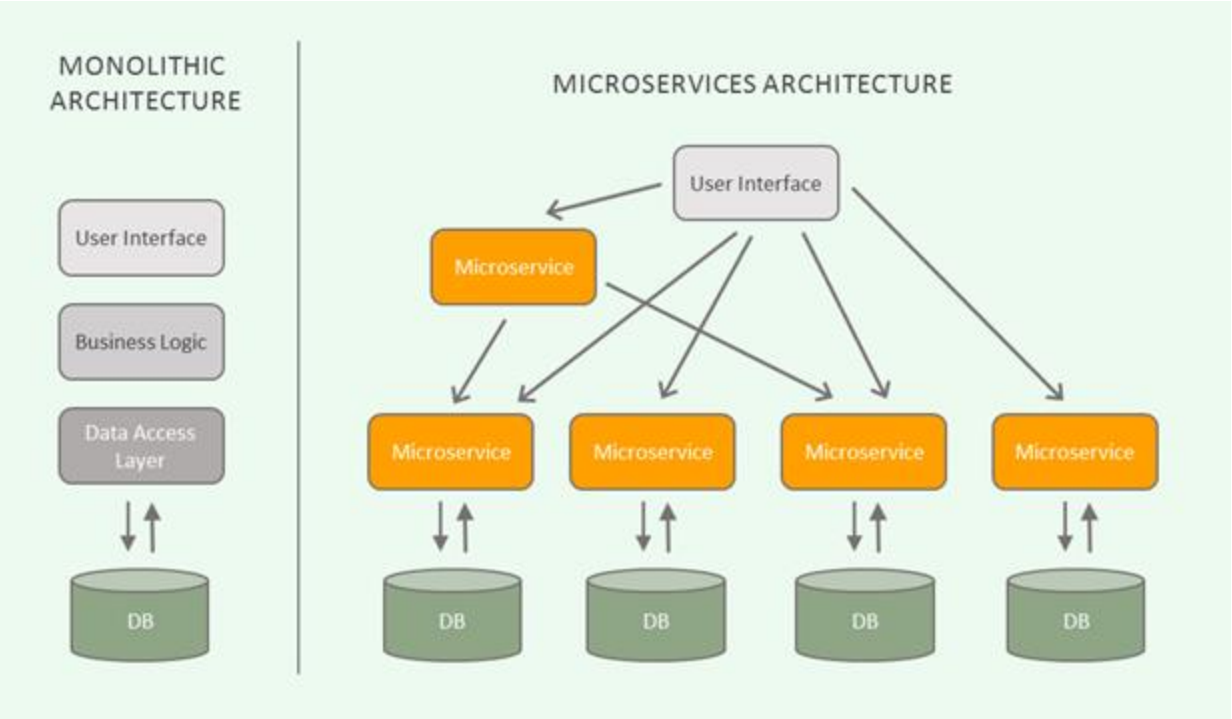
N-Tier architectures

- An extension of the three-tier architecture, where additional layers can be added to further separate concerns and improve scalability.
- Examples of additional layers: Service Layer (for handling API calls), Caching Layer (for improving performance), Security Layer, etc.
- Advantages: Highly modular, highly scalable, suitable for large, complex applications.
- Disadvantages: Increased complexity, can be challenging to develop and maintain.
- Use Cases: Large-scale enterprise applications, distributed systems, cloud-based applications.

Microservices

- An architecture style that structures an application as collection of services
- The execution of a single and specific functionality through Microservices Architecture framework permits developers to rollout applications faster and with greater efficiency.
- As various components are developed in different coding languages, there is greater flexibility in choosing a technology of choice.
- Examples: Netflix, Amazon, Uber, etc.

Microservices



Installations

- VS Code (https://code.visualstudio.com/docs/setup/windows)
- Node js (https://nodejs.org/en)
  - node –v (on terminal/command prompt)
- Npm